Numerical mathematics – computing with a computer

Most equations in physics and engineering cannot be solved exactly. Numerical mathematics is about finding approximate solutions – and knowing how good they are. In the 1970s calculations were typically run on the university's central computer and on the first programmable pocket calculators; today every phone has more computing power than all of Sussex's computers put together had then. The methods are the same. The page has an interactive comparison of methods for root finding and for differential equations.

> University of Sussex, Numerical Mathematics (1975–78): One of the mathematics courses in the physics syllabus. Ten years later I was to use the Newton–Raphson and Muller methods to find complex roots in my thesis at DTH. The page describes the subject as it is normally taught; it is not a reproduction of the original course.

Rounding errors

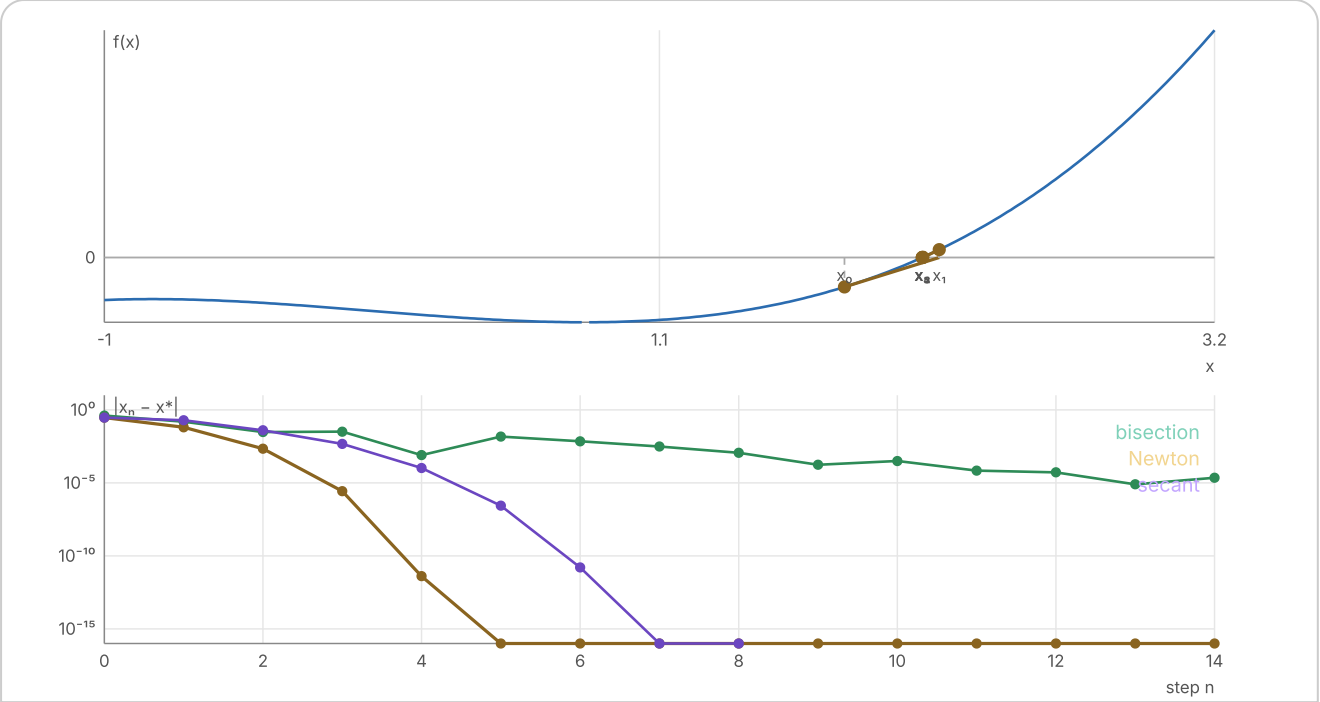
A computer calculates with a finite number of digits. Double-precision floating-point numbers (IEEE 754, standardised in 1985) have 53 binary digits, about 16 decimal places: the machine precision is $\varepsilon \approx 2.2 \cdot 10^{-16}$. That sounds like a lot, but errors can grow. Subtract two nearly equal numbers, and the leading digits vanish and the relative error explodes – *catastrophic cancellation*. The familiar formula for the roots of a quadratic therefore gives poor results when $b^2 \gg 4ac$, and it must be rewritten. The other big source of error is *truncation error*: an infinite process (a series, a limit) is stopped after finitely many steps. The art is to balance the two.

Root finding

To solve $f(x) = 0$ there is a range of iterative methods:

- **Bisection:** If an interval is known in which f changes sign, it is halved again and again. The method is slow – one binary digit per step – but it never fails.
- **Newton–Raphson:** The tangent at the current point is followed down to the x axis: $x_{n+1} = x_n - f(x_n)/f'(x_n)$. Near a simple root the convergence is *quadratic*: the number of correct digits doubles at every step. But a bad starting guess can send it far away, and at a double root it becomes slow.
- **The secant method:** Like Newton, but with a secant through the two latest points instead of the tangent – no derivative needed, and the order of convergence is the golden ratio, 1.618.
- **Muller's method** (1956) fits a parabola through three points and can find complex roots, even from real starting values.

Interactive root finding



Interpolation

If a function is known only at some points, a polynomial can be passed through them. With $n+1$ points there is exactly one polynomial of degree n (Lagrange or Newton form). But high degrees are dangerous: Carl Runge showed in 1901 that interpolating $1/(1+25x^2)$ at equally spaced points oscillates wildly near the ends, and that it gets worse the more points are taken. The cure is either to crowd the points towards the ends (Chebyshev points) or to use piecewise polynomials – cubic splines, named after the flexible strips shipbuilders used to draw hulls.

Numerical integration

An integral is approximated by a weighted sum of function values. The trapezium rule replaces the function with straight segments and has an error that falls as h^2, where h is the step length. Simpson's rule uses parabolas through three points and has error h^4 – halve the step, and the error becomes 16 times smaller. Gaussian quadrature chooses both points and weights optimally and integrates polynomials of degree $2n-1$ exactly with n points. Richardson extrapolation combines results with different h to remove the leading error terms – the basis of Romberg integration.

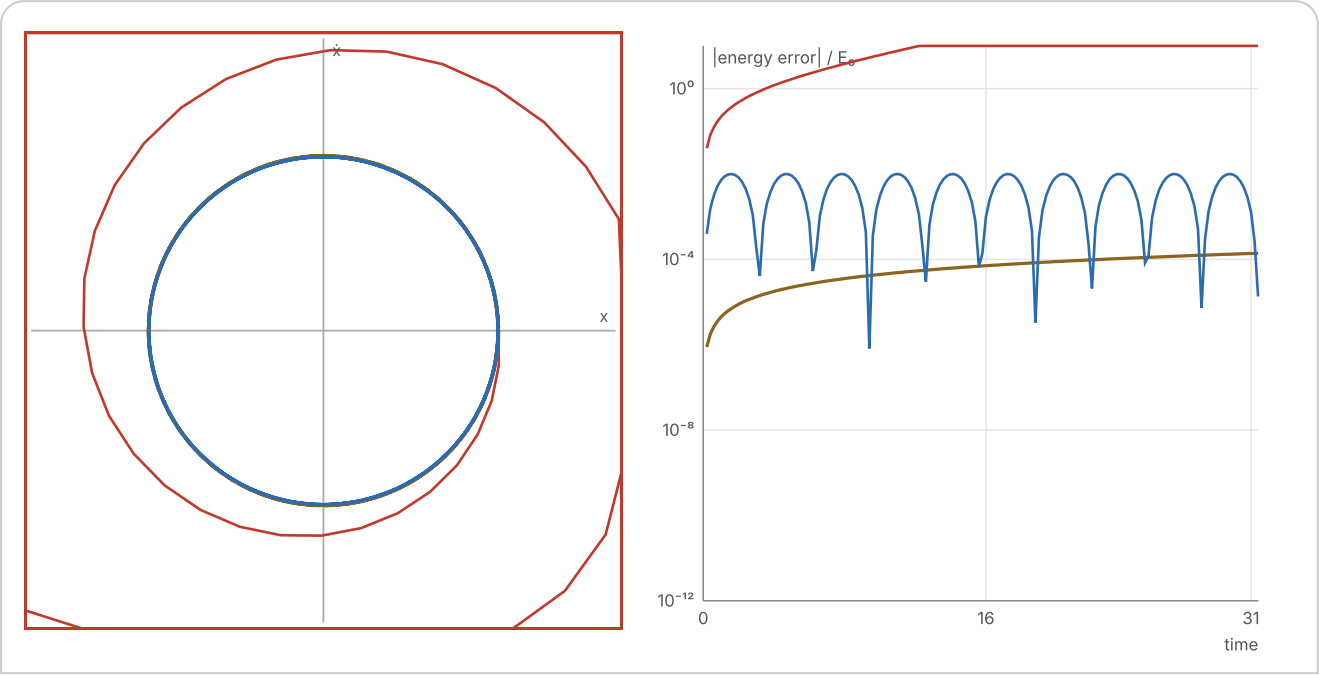
Linear systems

Gaussian elimination solves $A\vec{x}=\vec{b}$ in about $\tfrac{2}{3}n^3$ operations. To avoid dividing by small numbers, rows are swapped so that the largest element is on the diagonal – *partial pivoting*. The condition number $\kappa(A)=\|A\|\,\|A^{-1}\|$ says how much errors in the data are amplified: with $\kappa=10^{10}$ ten of

the sixteen digits are lost. Large, sparse systems from, say, discretised differential equations are better solved iteratively, with Jacobi, Gauss–Seidel or SOR (see the relaxation in applied field theory).

Differential equations

A differential equation $\dot{y} = f(t, y)$ is solved step by step. Euler's method $y_{n+1} = y_n + hf(t_n, y_n)$ follows the tangent and has a global error proportional to h. The fourth-order Runge–Kutta method (Runge 1895, Kutta 1901) estimates the slope four times per step and has error h^4 – it is the workhorse of scientific computing. For mechanical systems it is often more important to conserve energy than to be accurate in the short run: *symplectic* methods such as Störmer–Verlet (leapfrog) keep the energy error bounded for ever, whereas Euler lets it grow and RK4 lets it drift slowly.



Since 1978

- **IEEE 754** (1985) made floating-point numbers the same on all computers, so a program gives the same result everywhere.
- **Libraries:** LINPACK and EISPACK from the 1970s became LAPACK (1992), which now lies beneath MATLAB, NumPy and nearly all scientific software.
- **Adaptive solvers:** Modern ODE solvers (Dormand–Prince 1980, behind MATLAB's ode45) choose their own step length from an error estimate.
- **Graphics processors** have made parallel computing cheap – from climate models to the neural networks trained with variants of gradient descent.
- **Error analysis** has gained new importance with machine learning, where calculations are often done deliberately at low precision (16 or 8 bits) to save time and energy.

Further reading

- William H. Press et al.: *Numerical Recipes* (3rd ed., Cambridge 2007)
- Lloyd N. Trefethen and David Bau: *Numerical Linear Algebra* (SIAM 1997)
- Ernst Hairer, Christian Lubich and Gerhard Wanner: *Geometric Numerical Integration* (2nd ed., Springer 2006)

mhgfysik.dk · Michael H. Grant, lektor emeritus in physics & mathematics · The content may be reused under CC BY 4.0 with attribution. · My B.Sc. from Sussex · Home · Interactive figures: mhgfysik.dk/numerisk-matematik-en.html